Checking the Learned Dissipativity Conditions on the All-VSG Kundur Example: Where They Fail, Why, and a Structural Change That Fixes Most of It

Sehaj Randhir Singh, note for Dr. Andrey Gorbunov and Dr. Han Wang, October 2026. Simulation only. Built on the all-VSG Kundur example in Wang, Wang, Zhuang, Moffat and Dörfler, arXiv:2510.22324.

What this answers

Three questions. First, do the learned conditions hold along the closed loop, at $u = \pi(x)$, which is where Theorem 2 uses them, and not only at the training tuples? In my rebuild they don't. On five seeds the dissipativity inequality holds at the training tuples and is violated on 75% to 99% of the region X along the closed loop, and on 94% to 100% of states near the equilibrium.

Second, why? The cause is a mismatch of scale, and it can be measured. Near the equilibrium the inequality asks V to fall by 2.8% to 24% of its value in one 0.5 ms step, while a step delivers at most 0.007%, and the plant's own damping only allows about 0.03%. Φ was trained to be positive definite but nothing ties its size to the sampling step, and in discrete time it has to be of order Δt.

Third, does fixing that fix the conditions? Mostly. Making Φ positive definite by construction and training the inequality at $u = \pi(x)$ brings the violation down to 8% to 23% of X on all five seeds, with $\Phi \succ 0$ at every state. What is left sits in one specific place, at large angle and near-zero frequency deviation, and the paper's fault trajectory runs through exactly that band. That last point is the main thing I'd like your read on.

Part 1. The rebuild

I couldn't find code for the example, so I rebuilt it from the paper. The model is the Kundur two-area system with all four generators as VSGs, using the paper's parameters (virtual impedance j0.3 pu, H = 4 s, damping 5 pu on the 900 MVA VSG base). It's a Kron-reduced second-order swing model, $M\,d\omega/dt = P_{ref} - P_e(\delta) - D(\omega - 1) + u$ as in the paper's eq. (1), with a 0.5 ms zero-order-hold step and one B8-B9 circuit open after the fault. Without control it loses synchronism after the paper's fault, as in its Fig. 6. The controller follows the paper's parametrization: matrix networks $W(x)$, $Q(x)$, $S(x)$ and $R^{-1}(x)$ with $\pi(x) = -R^{-1}(x)S(x)^T x$ and $V(x) = x^T W(x) x$. It's trained with the paper's losses on 2,500 one-second trajectories (98,069 tuples), states relative to VSG1 and the region $X = \{|\Delta\delta| \le 2\pi,\ |\Delta\omega| \le 0.1 \text{ pu}\}$. I trained five seeds on the same data.

Each check uses 200,000 states sampled uniformly over X and 200,000 in each of two boxes at 5% and 1% of X (excluding the inner 5% of each box), plus 512-start gradient attacks on each condition. The inequality is checked as $F_D(x) = -x^T\Phi(x)x - (V(x^+) - V(x)) \ge 0$, since $s(x, \pi(x)) = -x^T\Phi x$ exactly.

Part 2. Why the training tuples can't see the failure

The inequality is trained on tuples where u is drawn uniformly over the input range. At those tuples the supply rate $s(x, u)$ is large, with a median of about 85 against a storage change of about 0.006 per step, so the inequality is easy to satisfy. Theorem 2 needs the closed loop, and at $u = \pi(x)$ the supply rate drops to its minimum over u, $-x^T\Phi(x)x$. That is the input with the least slack, and a uniform draw almost never lands near it. So a zero training loss on the tuples says very little about the case the proof actually uses.

Part 3. The scale mismatch

To first order in the step, $V(x^+) - V(x) = \Delta t \cdot \nabla V(x)^T f(x, \pi(x)) + O(\Delta t^2)$, so the decrease one step can deliver shrinks with Δt. The decrease the inequality asks for, $x^T\Phi(x)x$, doesn't depend on Δt at all. So for the inequality to hold, Φ has to be of order Δt times the continuous-time decrease rate. The plant sets a rough ceiling on that rate. With $D/2M = 45/144 = 0.31\ s^{-1}$ on the system base, the swing energy decays passively by about $2(D/2M)\Delta t \approx 3.1 \times 10^{-4}$ of itself per step. The controller can add to that, but with $|u| \le 0.3$ pu against $P_{ref} \approx 7$ pu it can't add orders of magnitude.

Near x* (box at 1% of X), median over states	Required decrease $x^T \Phi x$ / V	Delivered decrease (V − V^+) / V
Reproduced controller, five seeds	2.8% to 23.7% per step	0.0027% to 0.0071% per step
Restructured controller, five seeds	0.025% to 0.13% per step	0.23% to 0.27% per step
Passive budget, 2(D/2M)Δt	0.031% per step	

Table 1. The two sides of the inequality as fractions of V, 20,000 states per box. results/an/phi_budget_*.json.

The reproduced controllers demand 400 to 8,800 times more decrease per step than they deliver. The inequality isn't failing because the controller fails to stabilize locally. It fails because Φ, which sets the demand, was only asked to be positive definite and ended up two to three orders of magnitude too large for a 0.5 ms step on a plant with H = 4 s.

A first attempt shows why Φ has to be handled structurally rather than by penalty. Adding the inequality at u = π(x) to the loss, with nothing else changed, drove its violation to 0.01% of X within one epoch, but it got there by making Φ indefinite at every sampled state, which voids the theorem. Adding a $\Phi \succ 0$ penalty back didn't stop that within four epochs, because the two terms pull against each other.

Part 4. The structural change, five seeds

I kept the plant, region, dataset, original losses, network sizes, sampling and checks fixed, warm-started each seed from its own reproduced controller, and changed two things. The Q head now outputs a positive definite matrix P(x) and $Q := S R^{-1} S^T - P$, so $\Phi = S R^{-1} S^T - Q = P(x) \succ 0$ holds by construction. And the inequality and the closed-loop decrease are trained at u = π(x), on states drawn half over X and half in boxes at 10%, 3% and 1% of X, for 26 epochs.

Seed	Controller	Inequality violated, X	Same, 1% box	$V(x^+) \le V(x)$ violated, X	Φ not PD, X / 1% box	Worst F_D found by attack
0	reproduced	84%	100%	47%	23% / 0%	−9.60
	restructured	8%	12%	5%	0% / 0%	−0.32
1	reproduced	79%	96%	47%	31% / 100%	−11.3
	restructured	23%	37%	14%	0% / 0%	−0.26
2	reproduced	99%	100%	48%	2% / 0%	−6.69
	restructured	14%	24%	8%	0% / 0%	−0.31
3	reproduced	81%	100%	48%	28% / 0%	−7.62
	restructured	11%	19%	7%	0% / 0%	−0.27
4	reproduced	75%	94%	48%	37% / 100%	−6.83
	restructured	14%	30%	8%	0% / 0%	−0.28

Table 2. Share of 200,000 sampled states violating each condition, all with the training margin ε set to 0 (see Limits). Attack is 512-start projected gradient descent over X. results/seeds/recheck_*.json and verify_*.json.

The change helps on every seed, by a factor of 3 to 11 on the inequality over X and 3 to 8 near the equilibrium, and $\Phi \succ 0$ now holds everywhere instead of failing on up to 37% of X. The closed-loop decrease drops from 47% to 5% to 14%. The reproduced controllers sit at almost exactly 50% on that condition near x* on every seed, which is what you'd expect if the one-step change in V is close to zero and its sign is set by where the state is in its oscillation. After the change the required decrease is 0.11 to 0.53 of what a step delivers (median of the ratio, 1% box), against 10 to 86 before, so the mismatch from Part 3 is gone.

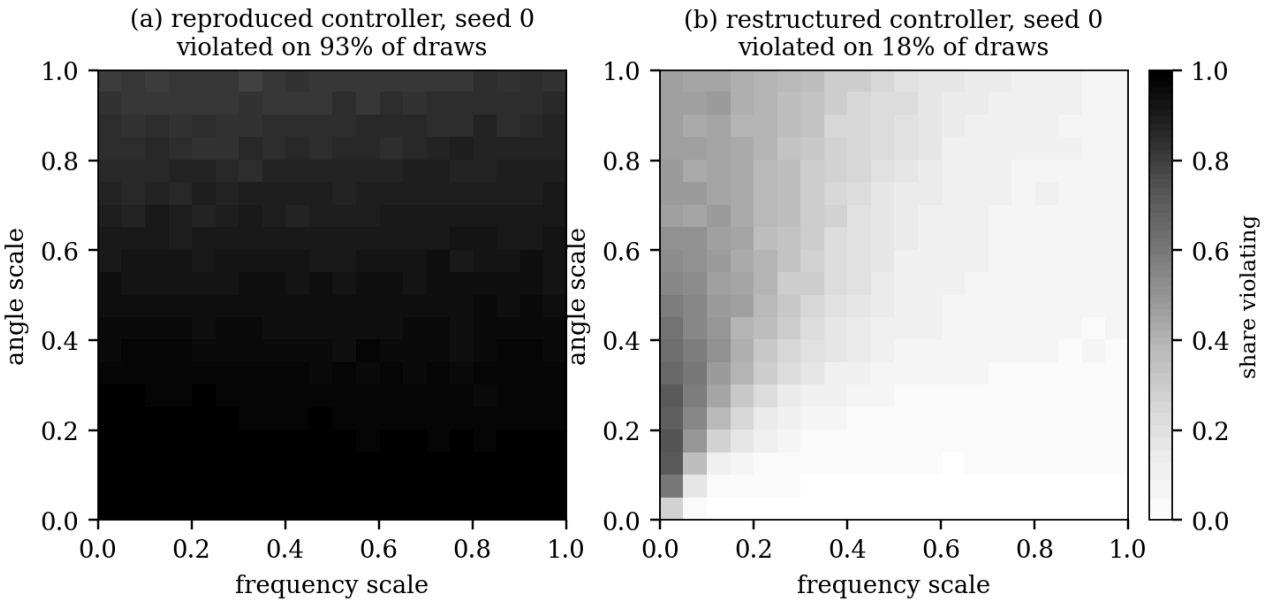


Figure 1. Share of draws violating the inequality, by how far out the state is in angle (vertical) and frequency (horizontal), each as a fraction of its half-width in X, 2,000 draws per cell. (a) Reproduced controller, violated nearly everywhere. (b) Restructured controller, where what's left sits in the left-hand band of low frequency deviation. Dense readout, not a certificate.

Part 5. Where what's left sits

The remaining violations aren't spread over the region. On every seed they concentrate at small frequency deviation and large angle (Figure 1b, Figure 2b). Over draws whose frequencies are within 20% of their half-width, 43% to 70% violate the inequality, against 5% to 14% when the frequencies are out past 80%. With every frequency deviation set exactly to zero, 47% to 76% of angle configurations violate it.

The swing model says why that band is hard. At $\Delta\omega = 0$ the angles don't move to first order in one step, since $d\delta/dt = \omega_B \Delta\omega = 0$, so the one-step change of $V = x^T W x$ comes only through the frequency rows, $V(x^+) - V(x) \approx 2\Delta t\, (Wx)_\omega^T\, \dot{\omega}$, and with $x_\omega = 0$ that is $2\Delta t\, (W_{\omega\delta} x_\delta)^T\, \dot{\omega}$. The decrease therefore has to come entirely from the angle-frequency cross block of W, while $x^T \Phi x = x_\delta^T \Phi_{\delta\delta} x_\delta$ is still strictly positive. In the trained networks that cross block is small, $\|W_{\omega\delta} x_\delta\|$ is a median 0.013 to 0.026 of the frequency diagonal times $\|x_\delta\|$ after restructuring, and 0.09 to 0.11 before. So the fine-tune shrank exactly the coupling this band needs.

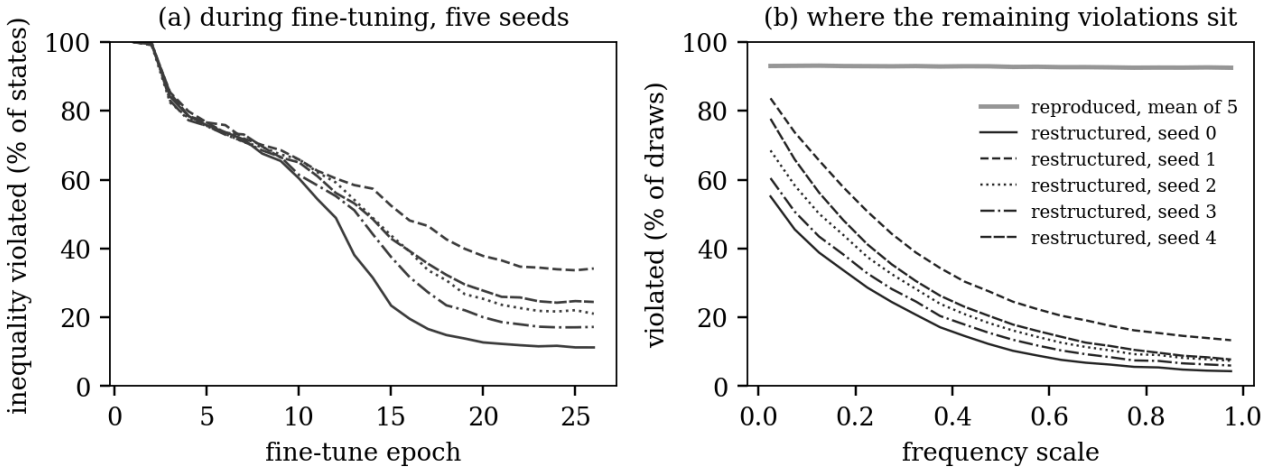


Figure 2. (a) Share of fresh states violating the inequality during fine-tuning, measured with the training normalization on the training mixture of region and boxes, so the endpoints are higher than Table 2. Seed 1 plateaus highest. (b) Violation rate against the frequency scale, averaged over angle scales, for the five restructured seeds and the mean of the five reproduced ones.

That leaves three explanations I can't yet tell apart. The first is the parametrization. A Φ that is uniformly positive definite asks for a strict per-step decrease even where the plant can only start decreasing V a step later, so either $\Phi_{\delta\delta}$ has to vanish as $\Delta\omega \to 0$ or the condition has to be read over several steps. The second is training. Seed 1 plateaus at 34% on the training measure against 11% for seed 0 with the same budget, so some of what's left is optimization. The third is the region. X allows $|\Delta\omega|$ up to 0.1 pu, which is 6 Hz, while the trajectories that matter use a small fraction of that, so uniform sampling over X spends most of its states at frequency deviations the grid never reaches. That links to the fault result below.

Part 6. The fault, and what the input does

After the paper's fault (B8 fault at 0.2 s, cleared at 0.32 s by tripping one B8-B9 circuit) every one of the ten controllers leaves X at 1.87 s to 1.88 s, the same as with no control (Figure 3a). My earlier guess was that the input bound decided this case, so I tested it. I re-ran the whole pipeline with $|u| \le 2.7$ pu on the system base, which is 0.3 pu on the 900 MVA VSG base and matches how I converted the damping. The restructured controller there violates the inequality on 8.5% of X, about as well as the best seed above, and it still leaves X at 1.88 s. The bound isn't binding. Before leaving X, the controllers use at most 0.007 to 0.026 pu of input with $|u| \le 0.3$, and 0.002 to 0.012 pu with $|u| \le 2.7$ (Figure 3b).

The reason is where the trajectory runs. From clearing until it leaves X, the largest frequency deviation averages 7% to 18% of the 0.1 pu half-width while the angles drift out to 2π. That is the low-frequency, large-angle band of Part 5, where the conditions are weakest and $\pi(x) = -R^{-1}S^{T}x$ is smallest, so the learned controller barely acts on the one trajectory the paper uses to show the enlarged region of attraction.

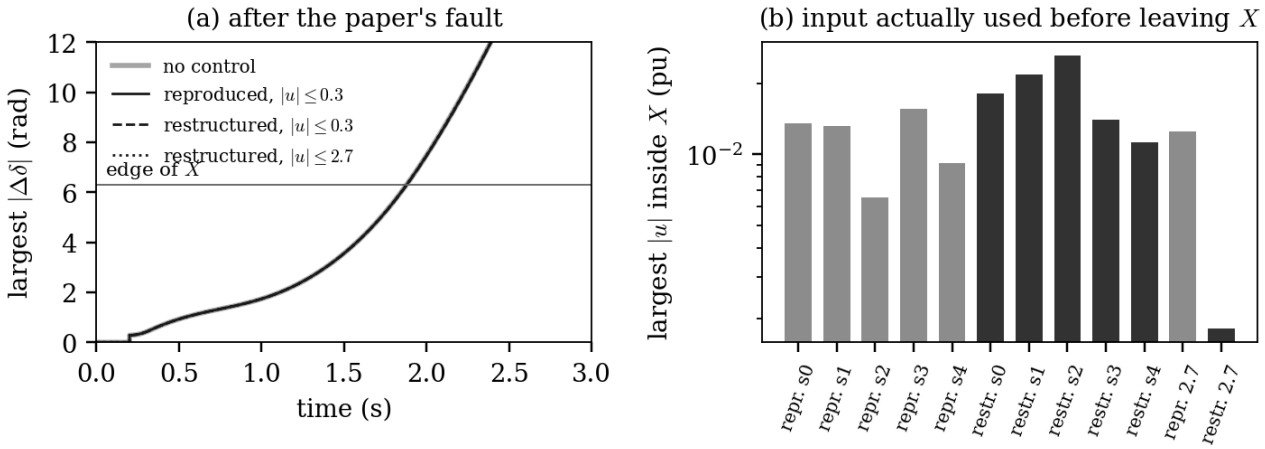


Figure 3. (a) Largest angle deviation after the paper's fault for no control and seed 0 at both input bounds. The curves lie on top of each other. (b) Largest |u| each controller applies before the trajectory leaves X, log scale. results/seeds/fault_seeds.json, results/u27/fault_u27.json.

How the cost grows

Step	Per seed	Wall time per seed
Reproduced controller, 60 epochs	98,069 tuples, batch 2,048	550 to 651 s
Restructure and fine-tune, 26 epochs	same tuples plus 2,048 closed-loop states per batch	361 to 438 s
Region check, dense plus attacks	600,000 states, 2,048 attack starts per condition	56 to 78 s

Table 3. Kaggle, 4 CPUs, seeds 1 to 4 and the |u| ≤ 2.7 run. Seed 0 ran on my laptop and took longer.

Where I'd like your read

Three things would change what I do next. Did you size Φ against Δt in any way, through normalization of the states, a scaling in the supply rate, or the choice of X, or is the paper's Φ only constrained to be positive definite? How was X sampled in training, and is $|\Delta\omega| \le 0.1$ pu the region you intended for the conditions or a filter for the data? And for the cost shaping in eq. (14), I used softplus(ε + s(x, π(x)) + l(x, π(x))) averaged over tuples, which gives a controller that barely acts on the fault trajectory, so I suspect I've read that term differently from you. If one of these explains the gap, I'd like to rerun the five seeds your way and send the comparison.

Limits, stated flatly

This is my rebuild with my own choices where the paper doesn't fix them (epsilons, the cost-shaping form, mean-plus-max aggregation, an input-range penalty, $|u| \le 0.3$ pu), not your controller, and your numbers could differ for any of those reasons. Every number here comes from dense sampling and gradient attacks, which find violations but can't prove their absence. I couldn't run a sound CROWN certificate because auto_LiRPA 0.7.2 doesn't support the GELU and softplus activations the networks use. The inequality is checked with ε = 0, because a constant margin of 10^{-3} can't hold as $x \to 0$ when both sides are around 10^{-6}. The input range $|\pi(x)| \le 0.3$ holds on every dense sample but the attacks find corners of X where it's exceeded, up to 0.52 pu, on 2 of 5 reproduced and 4 of 5 restructured seeds. Current limiting isn't modeled. The code, the trained controllers and every JSON behind these tables are ready to share.